`tests/regression/cache_cli_present_test.sh` — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Active standing regression guard (§11.4.135) for BUGFIX-CACHECLI (regression #50). **Authority:** Helix Constitution §11.4.135 (standing regression suite), §11.4.115 (RED_MODE polarity), §11.4.124 (dead/deleted-feature restoration), §11.4.7 (RED must reproduce), §11.4 (feature-layer PASS-bluff)

> Companion (§11.4.18) to the in-source documentation block at the top of the script. Pairs with the restored `cachectl` CLI.

Overview / Purpose

A standing regression guard proving the documented cache-management CLI is **present, executable, `bash -n`-parseable, and dispatches every documented subcommand**. Commit `6ec58ef` accidentally deleted the 368-line cache CLI (the tracked `cache` FILE collided with the gitignored `cache/` runtime DATA dir at the same path, so a broad `git add` recorded it as deleted), leaving a feature documented in README/USER_GUIDE/docs/CACHE.md/docs/TROUBLESHOOTING.md unusable while every green test stayed green (a §11.4 feature-layer PASS-bluff). The CLI was restored under the non-colliding name `cachectl`.

It does NOT grep-and-pass tautologically: it reads the REAL tracked `cachectl` file, `bash -n`-checks it, and verifies the `case "$1" in` dispatch alternation lists all seven documented subcommands as delimited tokens.

Usage

```
# GREEN guard (default) — cachectl must be present + complete:
tests/regression/cache_cli_present_test.sh           # exit 0 =
        PASS

# RED reproduce — the post-deletion state (CLI absent) must
        reproduce:
```

```
RED_MODE=1 tests/regression/cache_cli_present_test.sh # exit 0 =
        defect reproduced

# Runs automatically inside the suite (both polarities):
bash tests/run-tests.sh                                #
        test_regression_guards()
```

§11.4.115 RED_MODE polarity

`RED_MODE`	What it asserts	PASS means
`0` (default)	the restored `$PROJECT_ROOT/cachectl`	present + executable + `bash -n`-clean + dispatches all of `stats clear invalidate warmup list size trim` → the fix is present (GREEN guard)
`1`	a non-existent post-deletion CLI path	the CLI is ABSENT → the documented feature is unusable → the defect reproduces

A `RED_MODE=1` run that cannot reproduce (replica CLI unexpectedly present) is a §11.4.7 finding, not a pass.

Inputs

- **`RED_MODE`** (env, default `0`). No CLI args.
- Reads the tracked `cachectl` file at the repo root (GREEN); references a deliberately non-existent path (RED).

Outputs

- One `[PASS]`/`[FAIL]` verdict line + an `evidence:` path on stdout.
- An evidence file at `qa-results/regression/cachecli/cache_cli_present.<pid>.txt` (timestamp, `RED_MODE`, documented subcommand list, verdict, detail).
- Exit `0` = PASS, `1` = FAIL.

Side-effects

Creates `qa-results/regression/cachecli/` and writes one evidence file per run. No container, network, or live-cache access — inspects a tracked file only.

Dependencies

- `sh` (`set -eu`), `bash` (for `bash -n`), `grep`, `date`, `mkdir`.

Edge cases

- **CLI reverted / re-deleted** → GREEN guard reports `REGRESSION: cachectl feature broken -> file-absent;` (or the specific reason) and exits 1.
- **A documented subcommand silently dropped** → the dispatch-alternation check reports `missing-subcommands:[…]` and FAILs even though `bash -n` stays clean (the §1.1 paired-mutation behaviour).

Related scripts

- Restored feature: `cachectl` (was: tracked `cache`, deleted in `6ec58ef`) — see `docs/scripts/cachectl.md`.
- `tests/run-tests.sh` — registers this guard (GREEN + RED) via `test_regression_guards()`.
- `tests/comprehensive-test.sh` — exercises `./cachectl` command output live.

Last verified

2026-07-01 — documented from source; RED points at a non-existent CLI path and asserts absence, GREEN reads the real `cachectl` and asserts the 7-subcommand dispatch. `sh -n`/`bash -n` parse-clean per the script's own discipline. Not executed here.